

# CSE 4345: Big Data Analytics

Week 2, Part 1 — Challenges of Big Data: Extraction, Integration & Heterogeneous Information

Department of Computer Science & Engineering  
Premier University, Chittagong

Semester 2026

# Course & Lecture Information

## Lecture Identity

**Course:** CSE 4345  
**Week / Part:** 2 of 11 | Part 1 of 2  
**CLO:** CLO2  
**PLO:** PLO(b), PLO(c)

## CLO2

*“Analyse the existing challenges in Big Data”*

## Prerequisite Recall

Week 1 established the **3Vs**—Volume, Velocity, Variety. Week 2 dives into **Variety** in depth: what happens when you must integrate data from tens of heterogeneous sources.

## Part 1 Covers

- The data silo problem
- Four types of heterogeneity
- ETL: Extract, Transform, Load (in depth)
- ELT: The data lake paradigm
- Data quality dimensions & provenance
- Entity resolution
- Ivy League alignment
- Student assessment pool

## Part 2 Preview

Seminal paper: Halevy, Rajaraman & Ordille (2006)  
Case Study: HL7/FHIR & Healthcare Interoperability

# Lecture Agenda

- 1 The Data Silo Problem
- 2 Types of Data Heterogeneity
- 3 ETL and ELT Paradigms
- 4 Data Quality, Provenance & Entity Resolution
- 5 Ivy League Alignment
- 6 Student Assessment Pool

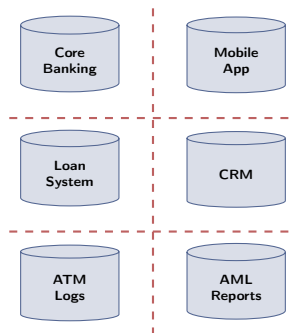
## The Data Silo Problem

# Organisations Are Data-Rich, Insight-Poor

## The Reality of Enterprise Data (2024)

A mid-sized bank in Bangladesh typically holds customer data in:

- **Core banking system** (Oracle RDBMS)
- **Mobile banking app** (MySQL + NoSQL)
- **Loan origination system** (separate legacy RDBMS)
- **Call centre CRM** (Salesforce cloud)
- **ATM transaction logs** (flat CSV files, nightly dump)
- **AML / fraud reports** (Excel spreadsheets)
- **Regulatory filings** (Bangladesh Bank XML schemas)



Data Silos — No Unified View

## A 2019 Forrester Study

**73%** of enterprise data is never analysed. The primary reason: **integration complexity**.

# The Integration Challenge at Scale

## The Core Problem Statement

*"Given a set of heterogeneous, autonomous data sources, provide the user with a **unified, transparent query interface** that gives a single coherent view of all data."*

—Halevy, Rajaraman & Ordille (2006)

## Three fundamental dimensions of difficulty:

- 1 **Heterogeneity:** Sources use different schemas, formats, vocabularies, and update cadences
- 2 **Autonomy:** Each source is independently managed; the integration layer cannot change the source
- 3 **Scale:** The volume and velocity of data make manual reconciliation impossible

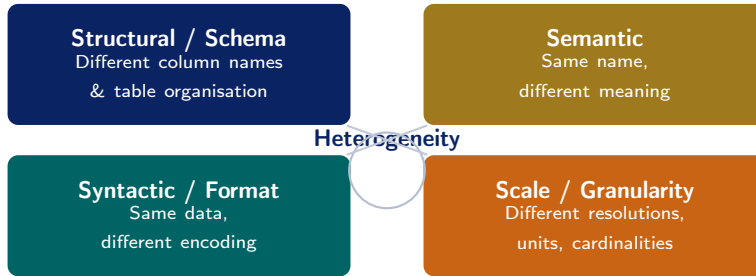
| Challenge     | Big Data Dimension                           |
|---------------|----------------------------------------------|
| Heterogeneity | Variety — different formats, schemas         |
| Scale         | Volume — 100 TB cannot be loaded into memory |
| Noise & gaps  | Veracity — sensor dropout, OCR errors        |
| Provenance    | Regulatory — GDPR, PDPA                      |
| Latency       | Velocity — real-time vs. nightly batch       |

## Key Insight

The 3Vs from Week 1 were symptoms. The

## Types of Data Heterogeneity

# Four Dimensions of Heterogeneity



## Why All Four Matter Simultaneously

A single integration pipeline must handle all four dimensions at once. Solving only structural heterogeneity (schema matching) while ignoring semantic heterogeneity (meaning) produces a technically unified but analytically **misleading** dataset.



# Structural (Schema) Heterogeneity

## Definition

Two data sources represent the **same real-world concept** using **different table structures, column names, or normalisation levels**.

## Two sub-types:

- **Naming conflicts:** Different names for the same attribute
  - Source A: `cust_id` ↔ Source B: `customer_number`
  - Source A: `dob` ↔ Source B: `birth_date`
- **Structural conflicts:** Flat vs. normalised schemas
  - Source A: one address column (full address as a string)
  - Source B: four columns (street, city,

## Bangladesh NID Integration Example

|            | BREC Schema            | MOHFW Schema               |
|------------|------------------------|----------------------------|
| ID field   | <code>nid_no</code>    | <code>national_id</code>   |
| Full name  | <code>full_name</code> | <code>fname + lname</code> |
| Birth date | <code>dob</code>       | <code>birth_dt</code>      |
| Address    | Single string          | 5 separate columns         |
| Gender     | M/F                    | 1/2                        |

Integrating Bangladesh Election Commission records with Ministry of Health records requires **mapping every field pair** before any query can span both.

# Semantic Heterogeneity

## Definition

Two data sources use the **same attribute name** (or structure) but **mean different things**, or use different vocabularies for the same concept.

## Common manifestations:

- **Homonyms:** Same word, different meaning
  - date = admission date in Hospital A
  - date = birth date in Hospital B
  - date = record-entry date in Health Ministry
- **Synonyms:** Different words, same concept
  - revenue vs. income vs. turnover (same KPI, different label)
- **Granularity mismatch:** “Region” = division in one system, district in another
- **Unit conflict:** Temperature in °C vs. °F

## Why Semantic Heterogeneity is Hardest

Structural heterogeneity can often be resolved **algorithmically** (match column names, infer types). Semantic heterogeneity requires **domain knowledge**—a machine cannot know that date means birth date without reading the data dictionary.

## Real Impact: Challenger Disaster (1986)

NASA and Morton Thiokol engineers used the word “acceptable” to mean different risk levels when discussing O-ring failures. This semantic mismatch contributed to the decision to launch.

*Source: Vaughan, D. (1996). The Challenger Launch Decision. U. of Chicago Press.*

# Syntactic & Scale Heterogeneity

## Syntactic / Format Heterogeneity

The **same data value** is encoded in **different formats** across sources.

Examples (all representing January 15, 2024):

| Format           | Value        |
|------------------|--------------|
| ISO 8601         | 2024-01-15   |
| US format        | 01/15/2024   |
| Bangladesh       | 15-01-2024   |
| Unix epoch       | 1705276800   |
| Bengali calendar | 01 Magh 1430 |

## Scale / Granularity Heterogeneity

Sources represent the **same domain** at **different levels of detail** or resolution.

Examples:

- **Geographic:** One source records GPS coordinates (lat/long to 6 decimal places); another records only the district name
- **Temporal:** Sales DB records hourly totals; ERP records daily totals; financial system records monthly summaries
- **Numeric precision:** Sensor A: temperature to 0.001°C; Sensor B: rounds to nearest integer
- **Aggregation:** One system reports

## Beyond Dates

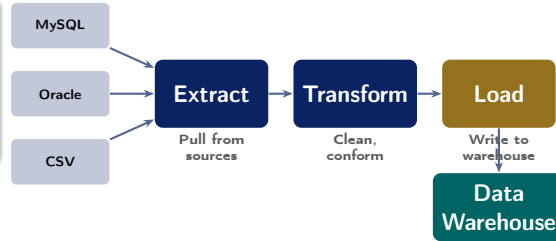
Phone numbers: 01711-123456 vs

## ETL and ELT Paradigms

# The ETL Process: Origin and Logic

## What is ETL?

**Extract – Transform – Load:** the classical approach to data integration, dominant from the 1990s through the 2010s. Data is **transformed before it reaches the warehouse**.



## Why ETL was designed this way:

- Storage was **expensive**—only clean, structured data deserved to be stored
- Data warehouses (Teradata, Netezza) had **rigid schemas** that required pre-conformed data
- Analysts knew *upfront* exactly which reports they needed—the schema was designed for known queries
- Regulatory compliance demanded **verified**,

## Key Characteristic

The data warehouse only ever sees **clean, structured, schema-conformed** data. Raw data never enters.

# ETL: The Three Phases in Depth

## Phase 1: Extract

Pull data from source systems:

- **Full extract:** dump entire source table (expensive, nightly)
- **Incremental extract:** only changed rows since last run (CDC — Change Data Capture)
- **Connectors:** JDBC for RDBMS, REST APIs, SFTP for file drops
- **Challenge:** source systems may be **unavailable** during extraction window

## Phase 2: Transform

The most **complex** stage:

- **Cleansing:** fix nulls, remove duplicates, correct typos
- **Standardisation:** unify date formats, currency, units
- **Schema mapping:** rename columns, restructure tables
- **Enrichment:** join with reference tables (e.g., add district name from postcode)
- **Aggregation:** compute summaries (daily totals from raw transactions)
- **Validation:** reject rows that violate business rules

## Phase 3: Load

Write transformed data to the target:

- **Full load:** overwrite the target table (simple, but slow)
- **Incremental load:** append or upsert new/changed rows
- **Slowly Changing Dimensions (SCD):** preserve historical versions of records (e.g., a customer's old address)
- **Partitioned load:** write to date/region partitions for query performance

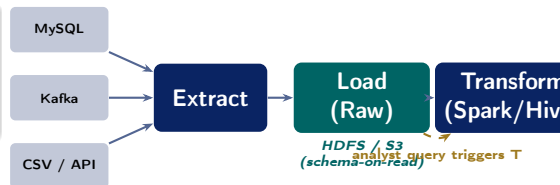
# ELT: The Data Lake Approach

## What is ELT?

**Extract – Load – Transform:** Load raw data *first* into a cheap, schema-flexible storage layer, then transform *on demand* using Spark or Hive.

## Why ELT emerged (post-2010):

- Cloud storage costs dropped to **\$0.023/GB/month** (AWS S3)—cheap enough to store raw data
- **Schema-on-read** (Hive, Spark): define schema at query time, not ingest time
- Data scientists need **raw, unprocessed** data for ML feature engineering—ETL discards too much
- Transformation requirements are often **unknown** at ingest time (exploratory analytics)



## Key Characteristic

Raw data **always preserved**. Transformation is triggered *by the query*, not at ingest time. Multiple analysts can apply different transformations to the same raw data.

# ETL vs. ELT: Comprehensive Comparison

| Dimension               | ETL                                                     | ELT                                                         |
|-------------------------|---------------------------------------------------------|-------------------------------------------------------------|
| <b>Transform timing</b> | Before loading (at ingest)                              | After loading (on query)                                    |
| <b>Schema</b>           | Schema-on-write (rigid)                                 | Schema-on-read (flexible)                                   |
| <b>Storage</b>          | Only clean data stored                                  | Raw <i>and</i> processed data stored                        |
| <b>Raw data</b>         | Discarded after transform                               | Always preserved in data lake                               |
| <b>Latency</b>          | High (transform pipeline runs before data is queryable) | Low (data available immediately after load)                 |
| <b>Best for</b>         | Known query patterns, compliance, sensitive data        | ML feature engineering, exploration, unknown future queries |
| <b>Tools</b>            | Informatica, SSIS, Talend, Ab Initio                    | Apache Spark + HDFS/S3, AWS Glue, dbt                       |
| <b>Example</b>          | Nightly bank reconciliation to                          | Netflix data lake: raw events                               |



## Data Quality, Provenance & Entity Resolution

# Data Quality: Six Dimensions (DAMA Framework)

| Dimension           | Definition                   | Big Data Example                                          |
|---------------------|------------------------------|-----------------------------------------------------------|
| <b>Accuracy</b>     | Correctly reflects reality   | GPS coordinates drifting 200 m                            |
| <b>Completeness</b> | All required data present    | 30% of sensor readings missing due to network dropout     |
| <b>Consistency</b>  | Data agrees across systems   | Customer age is 32 in CRM but 35 in loan system           |
| <b>Timeliness</b>   | Data is current for use case | Yesterday's stock price used for real-time trade decision |

## The “Garbage In, Garbage Out” Law

A sophisticated machine learning model trained on inaccurate, incomplete, or inconsistent data will produce **confidently wrong** predictions. Data quality validation must happen **before** any analytics.

## Cost of Poor Quality

IBM (2016) estimated that poor data quality costs the US economy **\$3.1 trillion per year** in lost productivity, regulatory penalties, and incorrect decisions.

*Source: IBM Big Data Hub, 2016*

# Provenance: Knowing Where Your Data Came From

## Definition of Data Provenance

**Data provenance** (also called *data lineage*) tracks the **origin, transformation history, and movement** of a data item from its source to its current form.

### Two types of provenance:

- **Where-provenance:** Which source record(s) produced this value?
  - e.g., This patient's age came from the NID database on 2024-01-10
- **How-provenance:** What transformations were applied?
  - e.g., `Salary field = gross_pay - tax - pf_deduction` (per Spark job v2.3.1, run 2024-01-15)

## GDPR Article 30: Mandatory Provenance

The EU General Data Protection Regulation requires organisations to maintain **Records of Processing Activities**—effectively a provenance log:

- Where did this personal data come from?
- Who has accessed it?
- What transformations were applied?
- Where was it sent (third parties)?
- When will it be deleted?

**Penalty for non-compliance:** Up to **4% of global annual revenue** or €20 million (whichever is higher).

# Entity Resolution: Linking Records Across Sources

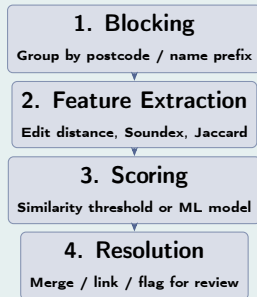
## Definition

**Entity resolution** (also: record linkage, deduplication, identity resolution) determines whether two records from **different sources** or **within one source** refer to the **same real-world entity**.

## Why it is hard:

- Name variations: “Mohammad Ali” vs. “Md. Ali” vs. “M. Ali”
- Typos: “Chittagong” vs. “Chattogram” vs. “Chittagram”
- Missing fields: no NID number in one source
- Identical names: 500 different “Rahim Uddin” records in a district

## Four-Step Entity Resolution Pipeline



## Blocking reduces complexity

Comparing only within blocks of records sharing the same first-3 letters of surname reduces  $O(n^2)$  to  $O(n \log n)$  in practice.

## Ivy League Alignment

# Data Integration in Top University Curricula

## MIT 6.830 Database Systems (Madden, Stonebraker)

MIT's graduate database course frames data integration as one of the **three unsolved grand challenges** in data management (alongside query optimisation and transaction management at scale).

- **LAV vs. GAV:** Two competing approaches to schema mapping
  - **GAV (Global-as-View):** Define global schema, write queries over sources
  - **LAV (Local-as-View):** Define each source as a view over the global schema
- **Pay-as-you-go integration** (Halevy, 2006): Integrate incrementally as queries arrive, rather than resolving all heterogeneity upfront

## Stanford CS246 Perspective

Leskovec et al. open their course with: *"Before you can mine data, you must **collect it, clean it, and integrate it.** In practice, 80% of a data scientist's time is spent on these steps."* **The 80/20 Rule of**

### Data Science:

- **80%** of effort: data collection, cleaning, integration
- **20%** of effort: modelling, analysis, visualisation

This is why CLO2 (*analyse integration challenges*) is foundational—you cannot reach CLO3/CLO4 without solving it.

## Student Assessment Pool

## Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

- Q1. Which data integration technique determines whether two records from **different datasets** refer to the same real-world entity?
- ☐ (a) Schema mapping    (b) Data exchange    (c) **Entity resolution**    (d) Data partitioning
- Q2. In the **ELT** paradigm, when does the transformation step occur?
- ☐ (a) Before data extraction    (b) During the extraction process    (c) Before loading into storage    (d) **After data is loaded, on demand at query time**

## Instructor Notes

**Q1:** “Data exchange” (option b) is a common confusion—it means materialising data from source to target schema, not linking records. **Q2:** Options (a) and (c) describe ETL; students must clearly distinguish ELT’s defining inversion.



- Q3. Which data quality dimension checks that each real-world entity is represented **exactly once** in a dataset?
- ☒ (a) Accuracy    (b) Completeness    (c) Consistency    (d) **Uniqueness**
- Q4. A healthcare database uses date to mean *admission date*, while a pharmacy database uses date to mean *prescription date*. What type of heterogeneity is this?
- ☒ (a) Structural heterogeneity    (b) Syntactic heterogeneity    (c) **Semantic heterogeneity**    (d) Scale heterogeneity

### Instructor Notes

**Q3:** Students often confuse uniqueness with consistency. Consistency = data agrees *across* systems; uniqueness = no duplicates *within or across* systems. **Q4:** The field name (date) is the same in both—this is a classic homonym/semantic conflict. Structural heterogeneity would be different column *names* for the same concept.

## Instructions

State True or False and provide a **one-sentence justification** for full marks.

| Statement                                                                                                             | Answer |
|-----------------------------------------------------------------------------------------------------------------------|--------|
| 1. ETL is the preferred paradigm for modern data lake architectures because it enforces schema early.                 | False  |
| 2. Semantic heterogeneity occurs when the same attribute name carries different meanings in different source systems. | True   |
| 3. In ELT, the transformation step is applied <i>before</i> data is written to the storage layer.                     | False  |
| 4. GDPR Article 30 effectively mandates that organisations maintain data provenance records for personal data.        | True   |

## Instructions

Answer each question in **100–150 words**. Use specific terminology from the lecture.

- Q1. Define the **ETL process** and explain each of its three phases (Extract, Transform, Load) with a concrete Big Data example from the banking or healthcare sector in Bangladesh.
- Q2. What is **schema heterogeneity**? Distinguish clearly between *structural* heterogeneity and *semantic* heterogeneity, and provide one specific example of each drawn from a multi-hospital patient data integration scenario.
- Q3. Identify and describe **three data quality dimensions** from the DAMA framework. For each, describe a concrete failure scenario in a Big Data pipeline and a mitigation strategy.

## Instructions

Answer in **200–300 words**. Show step-by-step reasoning and reference specific tools or concepts from the lecture.

### A1. Integration Architecture Design:

A Bangladeshi government agency wants to build a unified public health analytics platform by integrating data from 64 district hospitals, each using a different EHR system (some MySQL, some paper-based digitised as CSV, some using a custom XML API).

- Identify **four specific integration challenges** from today's lecture that this agency will face.
- For each challenge, propose a **concrete technical strategy** to address it.
- Should this agency use **ETL or ELT**? Justify with reference to their data characteristics.

### A2. Entity Resolution at Scale:

A national mobile financial services provider (like bKash) wants to merge its user database (50 million records) with Bangladesh Bank's credit history registry (40 million records). Explain why a naïve pairwise comparison is computationally infeasible, then design a **blocking strategy** and **similarity scoring approach** to perform this entity resolution at scale. What data quality dimension(s) would you validate *after* resolution is complete?

# Textbook & Reading References

## Primary Textbooks for Week 2

- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Chapter 2**
- **[TE]** Erl, Khattak & Buhler. *Big Data Fundamentals*, Prentice Hall, 2016. **Chapter 3**
- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed., 2020. **Chapter 1.3**

## Supplementary (Covered in Week 2, Part 2 — Seminal Papers)

- Halevy, A., Rajaraman, A., & Ordille, J. (2006). **Data Integration: The Teenage Years**. *Proceedings of VLDB*. **[PRIMARY SEMINAL PAPER]**
- Doan, A., Halevy, A., & Ives, Z. (2012). **Principles of Data Integration**. Morgan Kaufmann. *[Graduate textbook at MIT, Stanford]*
- Kleppmann, M. (2017). **Designing Data-Intensive Applications**. O'Reilly. Ch.10–11. *[Highly recommended supplementary reading]*

## For DAMA Data Quality Framework

DAMA International (2017). *DAMA DMBOK: Data Management Body of Knowledge*, 2nd ed., Ch.13 (Data

# Questions & Discussion

Week 2, Part 1 | CSE 4345 Big Data Analytics

---

*“Data is the new oil. But crude oil must be refined before it can power anything. Data integration is that refinery.”*

—Adapted from Clive Humby (2006)

Next Lecture: Week 2, Part 2

Seminal Paper: Halevy, Rajaraman & Ordille (2006) — *“Data Integration: The Teenage Years”*

Schema Mapping · LAV vs. GAV · Pay-As-You-Go Integration

Case Study: HL7/FHIR & Healthcare Interoperability (\$8.3B annual cost of failure)

**Reading before Part 2: [SA] Ch. 2 · [TE] Ch. 3**